Cloud-Based Life Safety Monitoring Platform

Technical Design Document (TDD)

1.Overview

This document describes the technical architecture, design decisions, runtime components, data flow, and implementation strategy for Cloud-Based Life Safety Monitoring System (NEWDOOR) cloud-based life safety monitoring platform.

The solution is designed to simulate, ingest, process, correlate, and broadcast operational safety events in real time across distributed cloud systems.

The platform demonstrates enterprise-grade architecture patterns including:

- Event-Driven Architecture
- Real-Time Stream Processing
- Distributed Runtime Execution
- Workflow Orchestration
- Intelligent Alert Correlation
- Scalable Cloud Messaging
- Real-Time Dashboard Broadcasting

The implementation focuses on the hackathon P1 scope while establishing a scalable enterprise-ready foundation

2.Disclaimer / Implementation Note

This implementation is delivered as part of a hackathon-style initiative with limited development timelines. **The current scope focuses on validating the core runtime architecture, ingestion pipeline, orchestration flow, and event-processing capabilities**.

Instead of integrating physical IoT devices, the solution introduces a Device Simulator application to generate ingestion traffic, telemetry events, heartbeat signals, offline scenarios, and incident events.

The Web Presentation layer leverages the existing DoWhatta.Platform.Web framework library developed by me. The framework provides reusable presentation foundations for web applications, API communication layers, shared web infrastructure, and theme injection capabilities to accelerate implementation.

The current implementation should be treated as a proof-of-concept runtime platform foundation for future enterprise-grade expansion.

3. Functional Scope

1. P1 Objective:

The NEWDOOR platform is designed to monitor and process real-time life safety events across distributed buildings and IoT devices.

Building Coverage

- 5+ Buildings

Device Coverage

- 5+ Devices per Building
- Smoke Sensors
- Heat Sensors

Supported Data Types

1. Normal Telemetry
    - Heartbeat
    - Device Health Status
    - Temperature Readings
    - Connectivity Status
2. Critical Events
    - Smoke Detected
    - Heat Spike Detected
    - Device Failure
    - Device Offline

Device Simulation

The platform includes a runtime device simulation engine to emulate real-world operational behavior.

- Periodic telemetry emission
- Real-time incident event generation
- Device online/offline simulation
- Intermittent network failure simulation
- Randomized telemetry spikes
- Configurable event frequency

Simulation	Description
Heartbeat Simulation	Periodic health event
Smoke Event Simulation	Critical smoke trigger
Heat Spike Simulation	High-temperature anomaly
Offline Simulation	Device stops sending heartbeat
Failure Simulation	Random intermittent failures

Cloud Ingestion Layer

The ingestion layer is designed to support scalable and reliable event intake.

- High-frequency telemetry handling
- Low-latency critical event ingestion
- Burst traffic support
- Concurrent device connection management
- Fault-tolerant event processing
- Guaranteed event delivery

Architecture Characteristics

- Event-driven ingestion
- Kafka-based streaming pipeline
- Async processing
- Stateless gateway services
- Horizontal scalability

Architecture Goals

- Loose coupling between services
- Real-time event processing

Runtime scalability

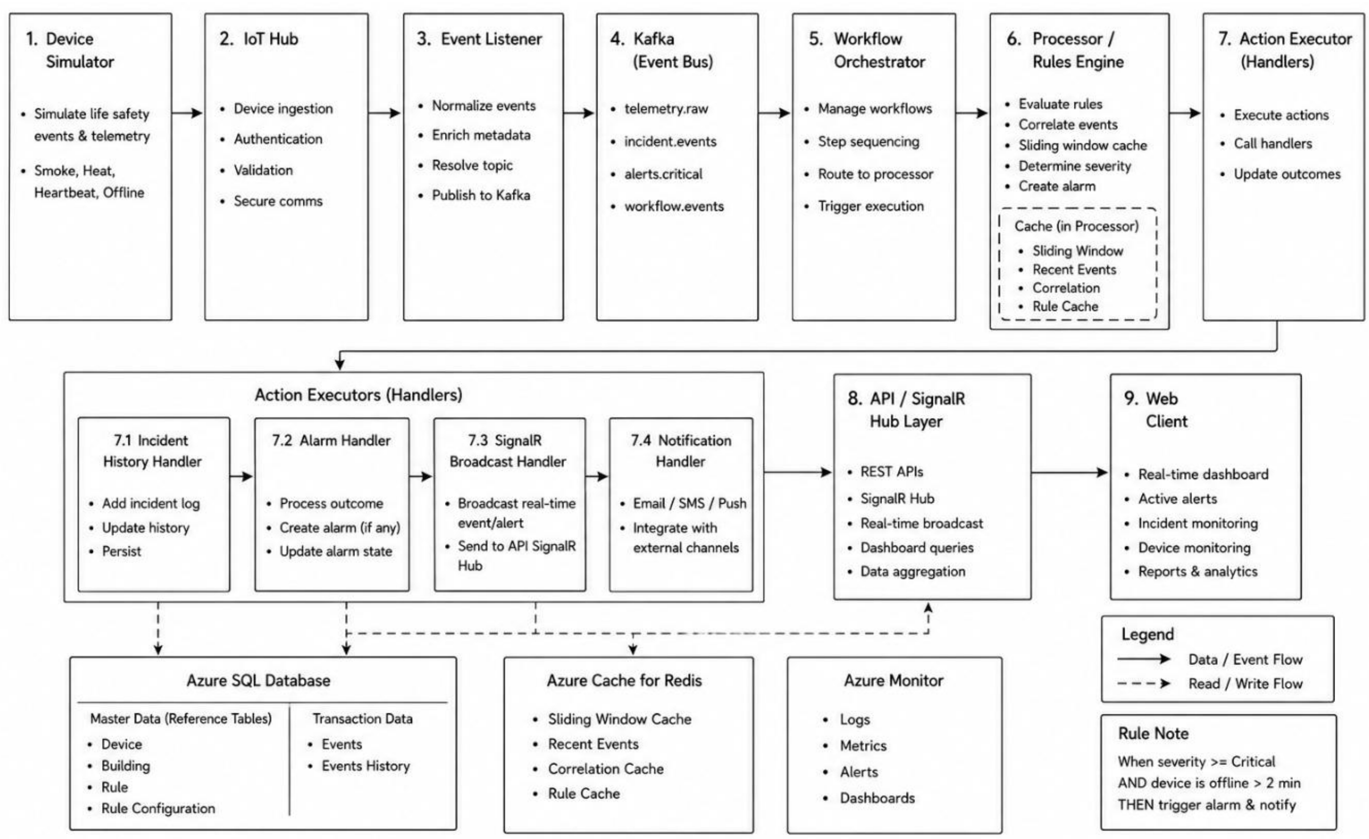
- Independent service deployment
- High-throughput ingestion
- Fault-tolerant processing
- Enterprise modular architecture
- Future SaaS extensibility

2. Architectural Decisions & Design Rationale

1.Core Principles

Principle	Description
Event-Driven	All operational activities are event-based
Cloud-Native	Built for distributed cloud infrastructure
Runtime-Oriented	Processing controlled through runtime pipelines
Loosely Coupled	Services communicate asynchronously
Horizontally Scalable	Services scale independently
Observable	Full monitoring and tracing support
Secure by Design	Security embedded into architecture
Extensible	New processors and workflows can be added dynamically

2. High-Level Architecture Flow



3. Component Architecture

NewDoor.Edge

- Gateway ingestion boundary
- Authentication handling
- Payload validation
- Device event intake

NewDoor.Client

- Operational dashboard
- Device simulator
- Runtime monitoring UI
- Incident visualization

NewDoor.Runtime

- Event normalization
- Runtime orchestration
- Rule evaluation
- Event correlation
- Alarm generation
- Action execution

NewDoor.Platform

- Messaging infrastructure
- Persistence layer
- Distributed cache
- Shared infrastructure services

NewDoor.ControlPlane

- Operational APIs
- Dashboard APIs
- SignalR hubs
- Runtime management endpoints

NewDoor.Core

- Domain entities
- Contracts
- DTOs
- Shared business models
- Common runtime abstractions

4. Core Runtime Flow

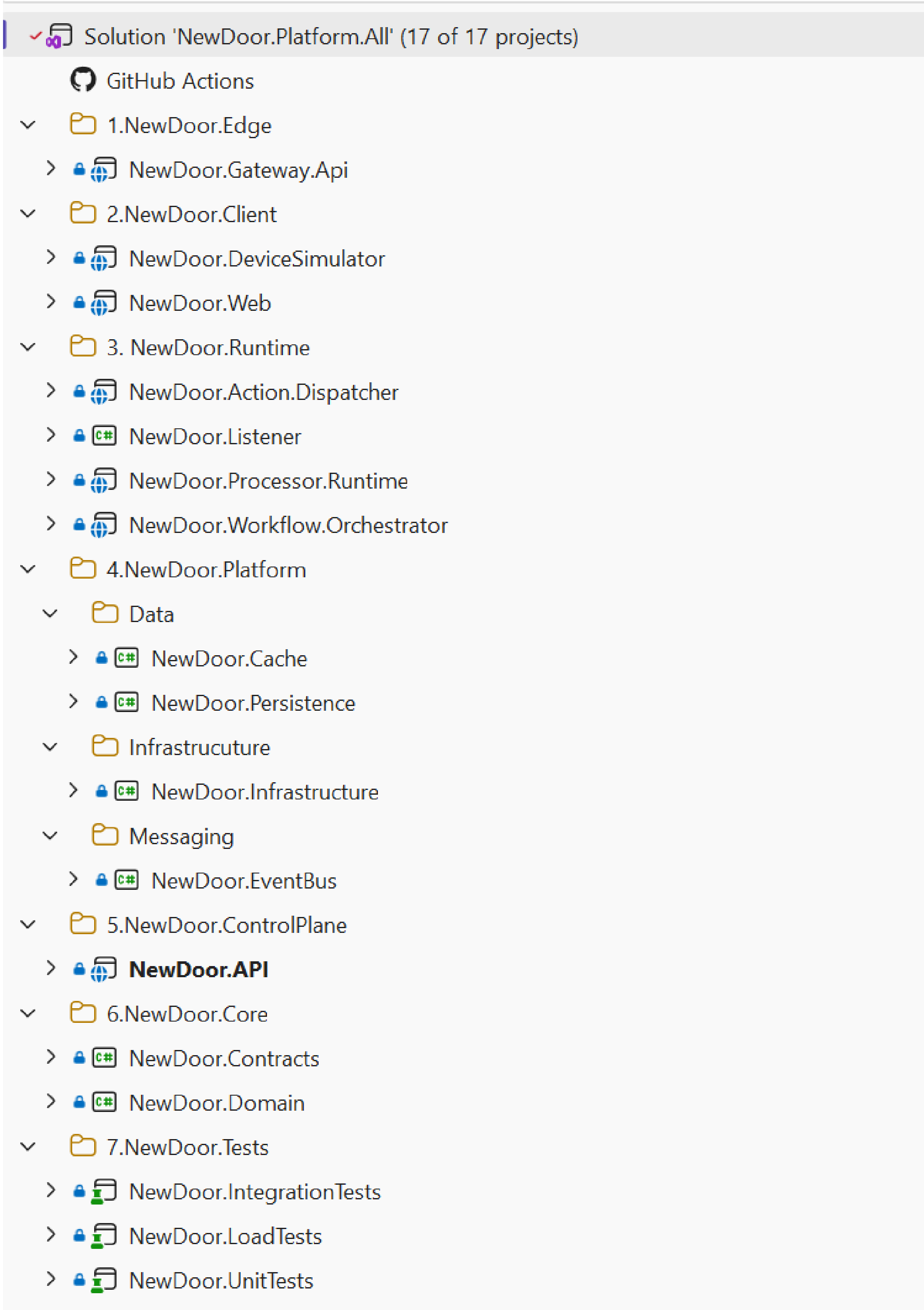
Core Runtime Flow processes device events through ingestion, normalization, rule evaluation, correlation, alarm generation, real-time broadcasting, and persistence.

```
Device Simulator
    ↓
newdoor.device.telemetry
    ↓
newdoor-event-processor
    ↓
newdoor.incident.detected
    ↓
newdoor-rule-engine
    ↓
newdoor.alarm.triggered
    ↓
newdoor-ui-service
    ↓
newdoor.ui.broadcast
    ↓
Blazor Dashboard
```

5. Technology Stack

#	Layer	Technology
1	Frontend	Blazor
2	Backend	ASP.NET Core
3	Messaging	Kafka
4	Database	SQL Server
5	ORM	Entity Framework Core
6	Cache	Redis
7	Realtime Communication	SignalR
8	Containerization	Docker
9	Runtime	.NET 8

4.Solution Architecture



5.Database Design

Master Entities

Table Name	Description
Device	Stores device master information and metadata
Building	Stores building details and hierarchy information
Rule	Stores runtime rule definitions
Rule Configuration	Stores configurable rule parameters and thresholds

Transactional Entities

Table Name	Description
Events	Stores incoming runtime device events
Events History	Stores processed and historical event records
Alarm	Stores generated alarms and alert details
Incident	Stores correlated incident information

6. Event Processing Strategy

1. Kafka Topics

Topic	Partitions	Purpose
newdoor.device.telemetry	12	High-volume device telemetry ingestion
newdoor.incident.detected	6	Incident events after telemetry processing
newdoor.alarm.triggered	3	High-priority alarms generated by rule engine
newdoor.ui.broadcast	2	Real-time dashboard broadcast events
newdoor.audit.history	3	Audit trail and normal event persistence

2. Consumer Groups

Consumer Group	Responsibility	Consumer Group
newdoor-event-processor	Processes telemetry events	newdoor-event-processor
newdoor-rule-engine	Evaluates incidents and applies business rules	newdoor-rule-engine
newdoor-ui-service	Pushes real-time SignalR updates	newdoor-ui-service
newdoor-audit-service	Stores audit and historical events	newdoor-audit-service

3. Component Consumer Mapping

Component Name	**Consumer Group**	**Consumes Topic**	**Produces Topic**	**Responsibility**
NewDoor.DeviceSimulator	—	—	newdoor.device.telemetry	Simulates device telemetry and incident events
NewDoor.EventProcessor	newdoor-event-processor	newdoor.device.telemetry	newdoor.incident.detected	Validates telemetry and creates incidents
NewDoor.RuleEngine	newdoor-rule-engine	newdoor.incident.detected	newdoor.alarm.triggered / newdoor.audit.history	Evaluates business rules and severity. High priority events generate alarms; normal events go to audit/history
NewDoor.RealtimeHub	newdoor-ui-service	newdoor.alarm.triggered	newdoor.ui.broadcast	Pushes real-time SignalR dashboard updates
NewDoor.AuditService	newdoor-audit-service	newdoor.audit.history	SQL Persistence	Stores audit history and compliance records
NewDoor.Web	—	newdoor.ui.broadcast	—	Displays live monitoring dashboard and alarms

4.High-Frequency Telemetry

Telemetry streams are processed asynchronously using Kafka consumer groups.

5.Low-Frequency Critical Events

Critical events receive prioritized runtime processing.

Reliability

Kafka ensures:

- Durable event streaming
- Retry capability
- Replay support
- Partition scalability

6. False Alarm Reduction Strategy

The system minimizes false positives using:

- Sliding window correlation
- Sustained anomaly checks
- Multi-sensor verification
- Event suppression logic
- Threshold-based escalation

Example:

- Single isolated heat spike → Suppressed
- Smoke + sustained heat spike → Critical Alarm

3. Future Roadmap (P2)

Future enterprise enhancements may include:

- Multi-tenant SaaS
- Rule designer
- Workflow builder
- AI anomaly detection
- Runtime replay engine
- Device provisioning
- Multi-site support
- Analytics platform

4. Conclusion

NEWDOOR establishes a scalable event-driven runtime platform focused on real-time life safety monitoring and distributed event processing.

The architecture emphasizes runtime isolation, modular platform design, scalable messaging, and real-time operational visibility while providing a strong foundation for future enterprise SaaS expansion.